

Teknisk dialog om RobotOps Twin

Diskussionsunderlag för Axel Kaliff och Per-Eric Olsson

Rami Halabi

Version 1.0 · 1 oktober 2026

Oberoende tekniskt rapportpaket · Svenska

Status: design och forskningsunderlag. Simulatorn beskrivs som ett planerat system. Inga empiriska robotresultat eller säkerhetsgarantier hävdas.

AI-assisterad text- och publiceringsframställning.

Ej sakkunniggranskad. Inte framtagen av eller godkänd av SICS AI.

Rapporternas källor, diagram och byggsript finns i projektets repository.

Innehåll

Så använder du underlaget	3
1. Din kärnberättelse på 90 sekunder	3
2. Ett bra upplägg för samtalet	3
3. Förklara arkitekturen utan att fastna i termer	4
4. Samtalsområden med Axel Kaliff	5
5. Samtalsområden med Per-Eric Olsson	7
6. Gemensamt whiteboardfall: kvittensen försvinner	9
7. Designbeslut du bör kunna försvara	10
8. Frågor du kan få och en bra svarsriktning	10
9. Frågor om rollen som projektet hjälper dig ställa	11
10. Det du ska ha redo före en faktisk demo	12
11. Samtalsprotokoll att fylla i	12
12. Avslutning	13
Referenser och källbegränsningar	13

Så använder du underlaget

Det här dokumentet är ett **arbetsunderlag för teknisk dialog**, inte ett manus som ska läsas upp ord för ord. Det hjälper projektägaren att presentera RobotOps Twin, förklara sina beslut och pröva antaganden tillsammans med Axel Kaliff och Per-Eric Olsson.

Version 1.0 gäller en specificerad men ännu inte verifierad simulator. Formuleringar som "jag har byggt", "testerna visar" eller "det fungerar" får användas först när de motsvarar körbar kod och sparade resultat. Fram till dess är "jag har specificerat", "jag föreslår" och "jag vill testa" rätt ord.

Samtalet ska visa förståelse, inte bara att många källor har lästs. En bra diskussion kan sluta med att ett eget designval behöver ändras. Det är ett användbart resultat.

1. Din kärnberättelse på 90 sekunder

Jag läste rollen som ett uppdrag att få AI, robotcell och kundsystem att fungera tillsammans, inte bara att koppla in en modell. Därför har jag specificerat en liten integrationssimulator där ett ERP/WMS skapar plockuppdrag och Blender ersätter den fysiska världen.

Jag har medvetet inte försökt återskapa er proprietära AI. Jag skiljer på kundens avsikt, ett modellförslag, ett robotkommando och den observerade effekten. Modell och robot är utbytbara gränssnitt, men deras garantier måste vara uttryckliga.

Huvudfallet jag vill testa är att ett plock genomförs men att kvittensen försvinner. Då ska systemet inte automatiskt plocka igen, utan visa att utfallet är okänt och stämma av journal och observation. Jag vill också visa ett fall där informationen inte räcker och systemet måste stanna för granskning.

Det jag gärna vill pröva med er är vilka av dessa gränser som motsvarar verkliga problem hos er, och vad jag har förenklat för mycket.

Bakgrunden till fokus på kundintegration finns i rollannonsen. Det betyder inte att simulatören uppfyller alla anställningskrav eller återger den verkliga implementationen. [\[S01\]](#)

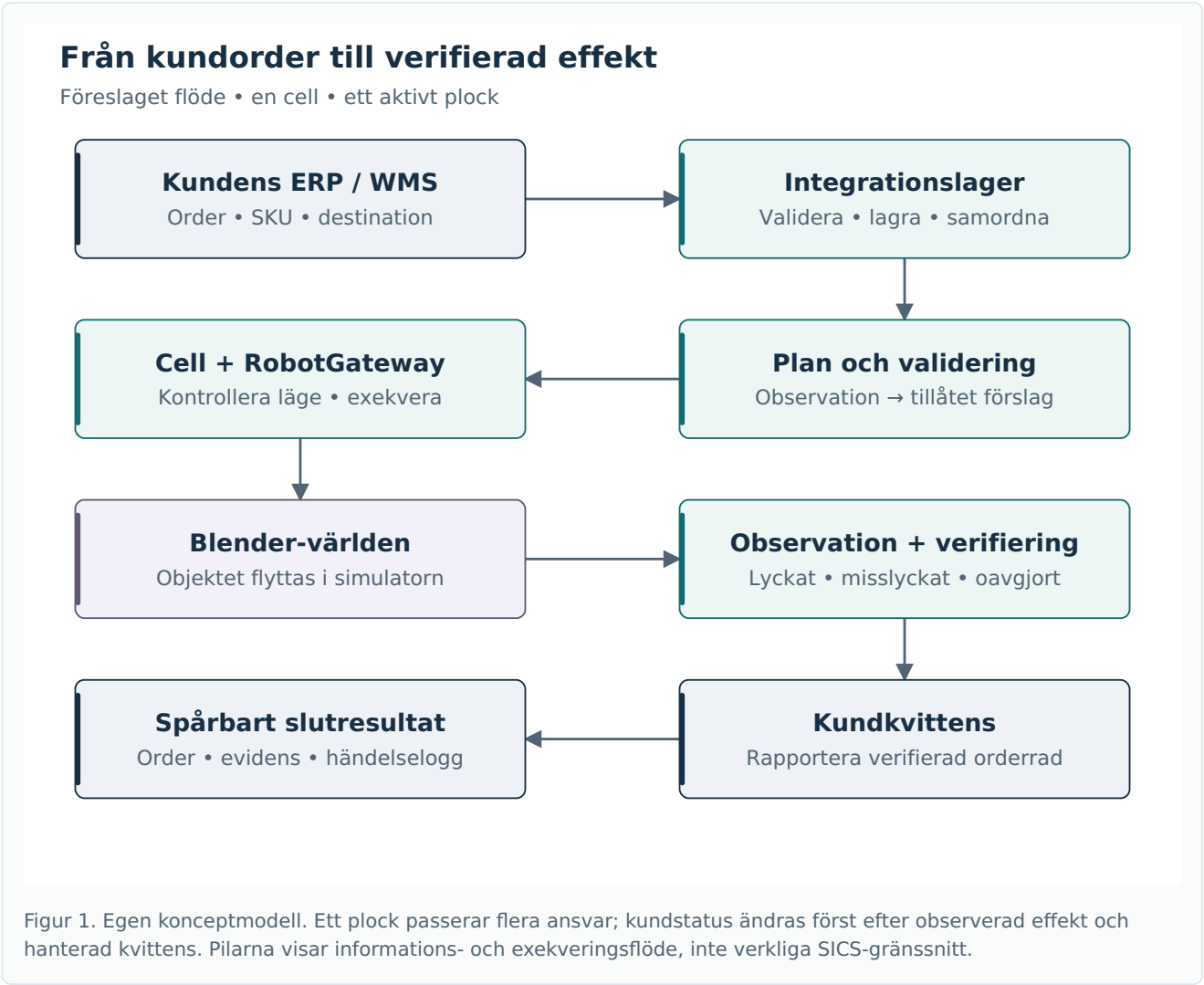
2. Ett bra upplägg för samtalet

Del	Vad du visar	Vad du vill lära dig
Öppning	Syftet och en ärlig statusruta.	Är problemet rätt valt?

Del	Vad du visar	Vad du vill lära dig
Arkitektur	Kund, integration, modell, cell, Blender och verifierare.	Var går motsvarande gränser hos dem?
Normalflöde	En order och dess planerade resa genom systemet.	Vilka kvittenser och observationer behövs?
Felväg	Förlorad kvittens efter effekt.	Hur hanterar de osäkra eller delvisa utfall?
Fördjupning	En designavvägning, inte tio verktygsnamn.	Vad skulle de ändra och varför?
Avslutning	Viktigaste lärdomen och nästa test.	Vad skulle vara ett relevant första arbetsresultat?

Planera en kärnpresentation på ungefär tio minuter, men följ deras frågor. Det är ett förslag till disposition, inte en uppgift om mötets faktiska längd eller intervjuformat.

3. Förklara arkitekturen utan att fastna i termer



En möjlig förklaring är:

ERP:t vet vilken order kunden vill ha plockad. Integrationslagret håller reda på uppdraget och dess historik. Brain-komponenten föreslår en handling utifrån en observation. Validatorn kontrollerar att förslaget passar aktuell cell och aktuellt tillstånd. Robotadaptorn utför kommandot i Blender. Verifieraren bedömer resultatet innan ERP:t får en slutlig kvittens.

Förbered en mening om varje ansvar. Börja inte med alla bibliotek, processer eller klassnamn. Om de frågar om implementation går du vidare till ett konkret kontrakt, en tillståndsövergång och ett test.

Tre skillnader som du ska kunna förklara

Accepterat är inte utfört. En robotcontroller kan ha tagit emot kommandot utan att ha avslutat det.

Utfört är inte lyckat. En rörelsesekvens kan vara färdig utan att rätt objekt hamnat rätt.

Verifierat är inte kundsynkat. Plocket kan vara korrekt medan uppdateringen av WMS fortfarande väntar. Då ska bara kommunikationen återförsökas, inte plocket.

Detta är resonemanget i vår design, inte påståenden om vilka fel SICS AI har i produktion.

4. Samtalsområden med Axel Kaliff

Axel är medförfattare till R900, och AutoGrasper-materialet beskriver separata roller för uppgift, datainsamling, återställning och felåterhämtning. Det gör dessa ämnen till relevanta, källgrundade ingångar. Det säger inte vilka frågor han kommer att ställa på en intervju. [\[S05\]](#) [\[S06\]](#)

4.1 Verifiering av faktisk uppgiftsframgång

Säg: "I min design är verifieraren en egen komponent. Jag vill inte att samma funktion som flyttar objektet också ska vara enda beviset för att uppgiften lyckades."

Fråga: "Vilken evidens brukar vara mest användbar för att skilja en avslutad rörelse från en lyckad uppgift? Hur hanterar ni ett resultat där sensorerna inte räcker för att avgöra?"

Följ upp: Be om ett avgränsat, icke-konfidentiellt exempel på falskt godkännande respektive falskt underkännande. Fråga om toleranserna beror på artikel, uppgift eller cell.

Ta med dig: Vilka observationer, tidsfönster och eftervillkor du bör modellera. Anta inte att en enda kamerabild räcker.

4.2 Ground truth och en begränsad observation

Säg: "Blender vet allt om scenen, men jag vill inte ge beslutsvägen samma tillgång. Test-oraklet får läsa facit; ordinarie verifiering ska arbeta med en begränsad observation."

Fråga: "Vilka gemensamma antaganden skulle ändå kunna göra mitt test missvisande? Hur skulle du konstruera ett scenario där verifieraren får fel trots att simulatorns metadata är konsekventa?"

Följ upp: Diskutera skymning, identitetsförväxling, gammal observation och fel kalibrering. Be inte bara om en bättre modell; försök förstå vilken informationslucka som orsakar felet.

Ta med dig: Minst ett testfall som inte kan lösas genom att bara läsa ett objekt-ID.

4.3 Återställning som en del av försöket

Säg: "AutoGrasper skiljer mellan normal uppgift, reset och recovery. Jag använder det som inspiration för egna tillstånd, inte som om det vore er produktionskod." [S05]

Fråga: "Vad brukar en automatisk reset behöva verifiera innan nästa episod startar? Vilka fel bör göra att en episod underkänns i stället för att återställas i tysthet?"

Följ upp: Hur undviks att misslyckade återställningar förorenar nästa försök? Ska övergången till nästa episod kräva känd objektplacering, känd robotpose eller båda?

Ta med dig: Ett konkret startvillkor och en loggpost som behövs för reproduktion.

4.4 Datainsamling och försöksdesign

Säg: "Jag vill spara misslyckade körningar lika noggrant som lyckade. Annars riskerar demonstrationen att bli ett urval av bra sekvenser."

Fråga: "Vilka metadata är lättast att glömma men viktigast när ett robotförsök senare ska förstås eller jämföras?"

Följ upp: Fråga om versionsspårning, kamerakalibrering, tidsstämplar, återställningsstatus och hur försöken delas upp vid utvärdering. R900 används här som inspiration för experimentdisciplin, inte som ett dataset vi har reproducerat. [S06]

Ta med dig: En minimal men användbar körningsmanifeststruktur.

4.5 Examensarbetets koppling: formulera dig försiktigt

En lämplig formulering är: "Jag såg examensarbetstiteln om Simulation In The Loop och action-success verification. Jag har inte kunnat granska hela fulltexten, så jag vill inte likställa min design med din metod. Vilken del av problemet tycker du är viktigast att skilja från en enkel efterhandskontroll?" [S04]

Det ger honom möjlighet att förklara skillnaden. Undvik att säga att examensarbetet visar att vår Blender-arkitektur fungerar eller att just hans metod har implementerats.

5. Samtalsområden med Per-Eric Olsson

Per-Eric presenteras av företaget som Chief AI Architect. HYPER-materialet ger en anledning att diskutera världsrepresentation, feedback och gränssnitt, men det ger inte insyn i alla tekniska detaljer. [S20] [S03]

5.1 Världsmodell kontra tillståndslagring

Säg: "Jag skiljer på simulatorns verkliga tillstånd, det observerade tillståndet, modellens interna representation och uppdragets mål. Min objektlista är ett integrationsformat; jag kallar den inte en reproduktion av er inlärd världsmodel."

Fråga: "På den nivå ni kan beskriva offentligt: vad ligger i modellens representation och vad ligger i vanlig systemlogik runt modellen?"

Följ upp: Hur hanteras tid, beständighet och osäker information? Hur märker integrationslagret att en plan bygger på gammalt tillstånd?

Ta med dig: En tydligare gräns mellan modellansvar och systemingenjörens ansvar, inte proprietära algoritmdetaljer.

5.2 Gränsen mellan uppgift och rörelse

Säg: "Jag har sett både beskrivningar av lednära styrning och av en separat motion-plattform. Därför har jag inte antagit att AI:n alltid returnerar en plockpunkt eller alltid en ledtrajektoria." [S03] [S07]

Fråga: "Vilken nivå bör en integrationsingenjör normalt tänka på: uppgiftsavsikt, greppose, bana eller tidsatta ledkommandon? Var valideras respektive nivå?"

Följ upp: Är samma gränssnitt relevant för alla robotar, eller finns profiler och kapabiliteter? Vem bestämmer när något ligger utanför en robots förmåga?

Ta med dig: Rätt abstraktionsnivå för MVP. Två möjliga kontrakt betyder inte att två fullständiga controllers måste byggas.

5.3 Feedback och ändrat beteende

Säg: "För en deterministisk baslinje kan jag återskapa logiken med samma version och indata. Om beteendet ändras under drift behöver jag veta vad som ändrats för att kunna felsöka."

Fråga: "Hur exponeras förändringar som är viktiga för drift: modellversion, representation, minne eller konfiguration? Vad behöver loggas för att en incident ska gå att återskapa?"

Följ upp: Finns en gräns mellan insamling, träning och godkänd modell för körning? Hur hanteras rollback och validering av ändrade beteenden?

Ta med dig: En begriplig livscykel för modell- och systemversioner. Anta inte att "kontinuerligt lärande" betyder att vikter uppdateras fritt efter varje plock.

5.4 Determinism, korrekthet och tidskrav

Säg: "Jag behandlar reproducerbarhet, rätt resultat och rätt tid som olika egenskaper. Ett system kan ge samma fel konsekvent."

Fråga: "När ni använder ordet deterministisk, vilken nivå syftar ni på: modellens inferens, planeringen, exekveringen eller tidsbeteendet?"

Följ upp: Vilka delar behöver hårda tidsgränser och vilka får vänta? Hur skiljer man en långsam modell från kötid, sensorfördröjning eller ett väntande cellvillkor?

Ta med dig: En faktisk tidsbudget och en tydlig terminologi, inte en abstrakt diskussion om huruvida AI "är deterministisk".

5.5 LLM/VLM runt den egna modellen

Säg: "Jag vill lägga en valfri modell bakom ett schema och låta den lämna ett förslag. Den får inte fri exekveringsrätt i Blender."

Fråga: "Vilka LLM/VLM-uppgifter ligger runt kärnmodellen i den här rollen? Är nyttan främst perception, tolkning av kunddata, diagnos, planering eller utvecklingsverktyg?"

Följ upp: Hur hanteras avvisade svar, hallucinering och otillräcklig evidens? Vad ska vara möjligt att köra utan extern modellanslutning?

Ta med dig: En konkret integrationsuppgift som motsvarar rollen, inte ett antagande att Codex ska ersätta deras robotbrain.

5.6 Compute utan att fastna i specifikationer

Fråga: "Vilka steg körs centralt och vilka måste fungera lokalt i cellen? Hur påverkar en förlorad nätverksförbindelse modellkörning och fortsatt drift?"

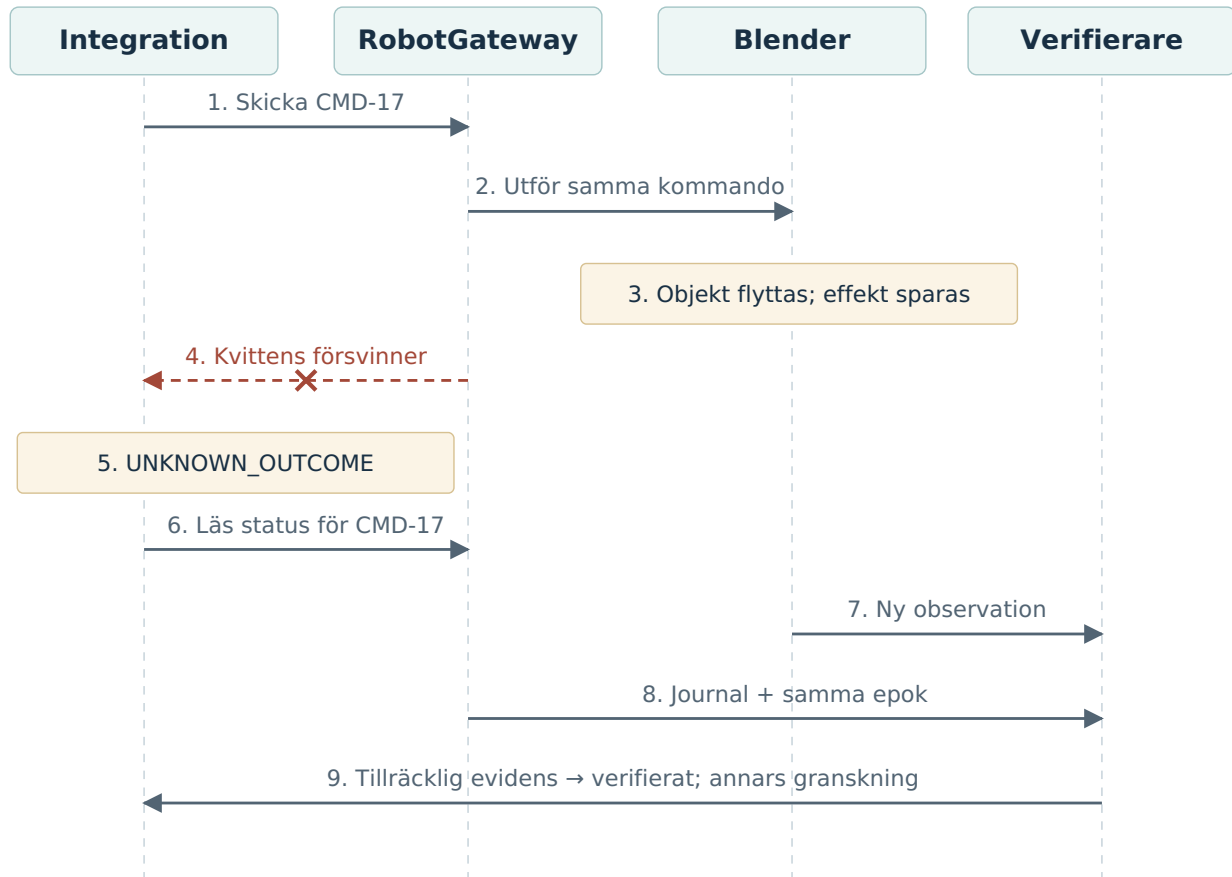
Följ upp: Fråga vem som äger paketering, drivrutiner, kapacitetsplanering och modellutrullning. LUMI- och DGX-källorna visar beräkningsarbete men besvarar inte rollens exakta ansvar. [\[S10\]](#) [\[S11\]](#)

Ta med dig: Gränsen mellan träningsinfrastruktur, inferenstjänst och cellnära integration. Du behöver inte lova att du kan administrera deras kluster utan att ha sett kraven.

6. Gemensamt whiteboardfall: kvittensen försvinner

Utfört plock — förlorad kvittens

Planerat felscenario T04 • inget empiriskt testresultat



Ingen ny PICK skickas under avstämningen.

Figur 4. Egen sekvensmodell. Effekten uppstår före kommunikationsfelet. Integrationen begär kommandostatus och observation, inte ett nytt plock. Endast tillräcklig och konsistent evidens tillåter slutförande; annars krävs fortsatt utredning.

Beskriv först bara observationen: "Klienten fick timeout efter ett plockkommando." Be dem sedan resonera tillsammans med dig om vilka händelser som kan ligga bakom.

Ditt eget förslag:

Jag registrerar avsikten före sändning och använder ett stabilt kommandokontrakt. Efter timeout vet jag inte om effekten uteblev. Jag vill fråga efter samma kommando och ta en ny observation. Om journalen är konsistent och objektet verifieras i destinationen behöver jag inte utföra plocket igen. Om underlaget är motstridigt eller saknas ska ärendet bli synligt oavgjort.

Bra följdfrågor är: "Hur länge behåller kontrollern kommandostatus?", "Vad betyder not found efter omstart?", "Kan ett objekt ligga kvar i griparen?" och "Vad krävs för att ett nytt försök ska vara tillåtet?"

Undvik att lova exactly-once utan att definiera vilken gräns och felmodell garantin gäller för. Poängen är inte att vinna en diskussion om terminologi, utan att undvika en ogrundad fysisk sidoeffekt.

7. Designbeslut du bör kunna försvara

Beslut	Ditt motiv	Kostnad eller begränsning
En cell och ett aktivt plock	Gör ägarskap och felvägar begripliga.	Ingen flerrobotskalning demonstreras.
SQLite i första versionen	Liten, beständig lokal miljö.	Inte bevis för distribuerad hög tillgänglighet.
Separat kundkvittens	Skiljer fysisk effekt från affärssystemets uppdatering.	Fler tillstånd och fler tester.
Deterministisk Brain-baslinje	Gör E2E-fel reproducerbara utan modellkonto.	Visar inte egen inlärdd robotpolicy.
Begränsad Blender-runtime	Tydliga operationer och felsökningsbarhet.	Mindre frihet än godtycklig agentkod.
MCP bara vid utveckling	Skiljer kraftfulla författarverktyg från exekveringskontrakt.	Två separata arbetssätt måste dokumenteras.
Verifierare och test-orakel	Gör felaktiga slutsatser möjliga att mäta.	Delade simulatorantaganden kvarstår.
UNKNOWN som eget tillstånd	Undviker att gissa vid otillräcklig evidens.	Kräver ett arbetssätt för granskning och återhämtning.

Fråga gärna vilket av dessa val de först skulle ändra. Ett bra svar från dig kan vara: "Det verkar rimligt; då behöver jag ändra det här kontraktet och lägga till det här testfallet."

8. Frågor du kan få och en bra svarsriktning

"Varför Blender och inte en full robotsimulator?"

"Jag optimerar första versionen för att synliggöra integrationsflödet och felvägarna. Jag gör inga påståenden om realistiska kontaktkrafter. Om frågan blir fysik, styrengrannhet eller sim-to-real behöver simulatorvalet och testmodellen omprövas."

"Är detta verkligen AI?"

"Baslinjen är deterministisk planeringslogik. Den är medvetet inte ett forskningsbidrag inom robotinlärning. AI-adaptern är valfri och får lämna strukturerade förslag. Det jag vill demonstrera är systemansvaret runt en sådan komponent."

"Hur vet du att plocket lyckades?"

"Jag behöver en definierad framgångsregel och observation efter handling. I test kan ett separat orakel kontrollera om verifieraren gjorde rätt. I verklig utrustning måste sensor- och verifieringskedjan valideras på nytt."

"Varför inte bara retry?"

"För att samma timeout kan betyda att ingenting hände eller att handlingen redan utfördes. Jag börjar med att bestämma vad mottagaren garanterar och hur utfallet kan stämmas av."

"Vad händer om modellen returnerar ett giltigt men felaktigt kommando?"

"Schema är bara första kontrollen. Förslaget måste också passa aktuell observation, objektidentitet, celltillstånd, kalibrering och tillåtna operationer. Även efter det kan uppgiften misslyckas, så resultatet behöver verifieras."

"Hur mycket av detta gjorde AI åt dig?"

"Jag använder AI för utkast och implementation, men ansvarar för att förstå och granska ändringarna. Jag ska kunna förklara den kod jag visar, vilka antaganden den gör och vilka tester som faktiskt har körts."

Visa gärna en konkret diff, ett test som först misslyckades och hur du verifierade ändringen. Säg inte att du har gjort detta om det ännu inte finns.

"Kan vi byta Blender mot vår robot direkt?"

"Inte utan nytt integrationsarbete. Affärsgränsen är avsiktligt separerad, men adapter, tidskrav, kalibrering, säkerhetsansvar och hårdvarutester måste specificeras för er miljö."

9. Frågor om rollen som projektet hjälper dig ställa

Be om ett konkret exempel på ett första uppdrag: vilket kundflöde som ska fungera, vilka system som finns och vem som accepterar leveransen. Det är mer användbart än en allmän fråga om teknikstack.

Fråga också hur ansvaret delas mellan modellutveckling, integration, robotleverantör och drift. Vem prioriterar när en kundanpassning konkurrerar med plattformarbete? Vilken hjälp finns inom områden som är nya för dig? Vilka delar förväntas du äga själv efter tre månader?

En bra avslutande rollfråga är:

”Vilken av de här integrationsfrågorna skulle ni vilja att jag kan lösa självständigt först, och hur skulle ni avgöra att lösningen är tillräckligt bra för kund?”

Detta är frågor om arbetsinnehåll och leveransförväntningar, inte ett löfte från dig om kompetens som du ännu inte kan belägga.

10. Det du ska ha redo före en faktisk demo

Förbered en ärlig statusruta med implementerat, simulerat, testat och återstående. Ha versionsnumret synligt. Visa ett litet dataexempel, en tillståndsovergång och minst ett relevant feltest. Rensa bort nycklar och privata kunddata från skärmen och terminalhistoriken som du visar.

Om programmet fungerar: skapa ordern live, följ ID:t genom systemet och visa Blender-effekten. Visa sedan kvittensbortfall och ett oavgjort fall. Om programmet inte är färdigt: använd diagrammen och säg att det är designgranskning. En inspelad sekvens ska tydligt presenteras som inspelad.

Ha en reservversion av rapporterna som PDF. Börja inte installera nya drivrutiner eller flytta runtime till en okänd miljö precis före presentationen. Det är en praktisk rekommendation för denna demo, inte en uppgift om SICS interna rutiner.

11. Samtalsprotokoll att fylla i

Fråga eller antagande	Vad de faktiskt sade	Designkonsekvens	Nästa verifiering
Var slutar modellens ansvar?			
Vad räknas som lyckat plock?			
Vilka fel är vanligast?			
Vad lagras vid en omstart?			
Vilka observationer är tillförlitliga?			
Vilka protokoll används i relevant cell?			
Vad bör MVP inte försöka göra?			

Fråga eller antagande	Vad de faktiskt sade	Designkonsekvens	Nästa verifiering
Vad är en rimlig första leverans i rollen?			

Anteckna deras faktiska formuleringar och skilj dem från din tolkning. Publicera inte nya mötesuppgifter eller interna detaljer i det offentliga repot utan att först stämma av vad som får delas.

12. Avslutning

En möjlig sammanfattning efter samtalet är:

Jag ville använda projektet för att göra mina antaganden synliga, inte för att låtsas känna er implementation. Det viktigaste jag tar med mig är var ni sätter gränsen mellan modell, integration och robot, och vilken evidens som behövs för att ett uppdrag ska räknas som lyckat. Nästa steg för mig är att pröva de antagandena med ett litet reproducerbart test.

Det starkaste du kan visa är inte att du redan kan allt. Det är att du kan avgränsa ett problem, formulera ett testbart kontrakt, förklara en avvägning och ändra uppfattning när bättre information kommer fram.

Referenser och källbegränsningar

Referenserna avser avgränsade sakuppgifter. Diagram, kontrakt och testplan är egna förslag. Gemensamt kontrolldatum: 1 oktober 2026.

[S01] Systemingenjör — annons 4464336388

Arbetsgivarens rollbeskrivning

Superintelligence Computing Systems. Utan säkert publiceringsdatum. Kontrollerad 2026-10-01.

<https://se.linkedin.com/jobs/view/systemingenj%C3%B6r-at-superintelligence-computing-systems-4464336388>

Offentligt sökindexerad annons kontrollerad. Används som kravunderlag; inte som bevis för intern teknikstack. Direktsidan kunde inte återläsas konsekvent.

[S03] Foundation AGI model for Industrial Robots — HYPER: reporting

Självrapporterat projektarbete

SICS AI, deltagarrapport hos Europeiska kommissionen/CORDIS. Uppdaterad 1 juni 2026. Kontrollerad 2026-10-01.

<https://cordis.europa.eu/project/id/101218436/reporting>

Projekt-ID 101218436. Beskrivningen av modell och robotkontroll är deltagarens redovisning, inte oberoende reproduktion. DOI för projektet: 10.3030/101218436.

[S04] Simulation In The Loop: Real-Time Robot Action Success Verification with a Digital Twin

Examensarbete — bibliografisk uppgift

Axel Kaliff, KTH. 2025. Kontrollerad 2026-10-01.

<https://kth.diva-portal.org/smash/record.jsf?pid=diva2%3A1988006>

Titel och författarsamband stöds av examinatorns offentligt indexerade KTH-sida. DiVA-fulltexten kunde inte hämtas vid denna kontroll. Inga detaljerade metod- eller resultatpåståenden återges.

Kompletterande primärkälla: <https://www.csc.kth.se/~fpokorny/>

[S05] Project Autograsper: Custom Grasper Development Guide

Öppen primärdokumentation

CloudGripper-projektet. Löpande repositorydokumentation. Kontrollerad 2026-10-01.

<https://github.com/cloudgripper/autograsper/blob/main/README.md>

README granskad via GitHub. Git-blob: 3cb1da256bc5147ef52a56327da476f110c4e6c2. Forskningsverktyg; inte SICS AI:s produktkod. Ingen kod eller fjärrrobotkörning har återanvänts i rapportpaketet.

[S06] R900: Understanding the Cost-Effectiveness of Random Exploration from 900 Hours of Robotic Data Collection

Forskningspreprint

Shutong Jin, Axel Kaliff, Ruiyu Wang, Muhammad Zahid och Florian T. Pokorny. arXiv v2, 17 september 2025. Kontrollerad 2026-10-01.

<https://arxiv.org/abs/2503.23571>

DOI: 10.48550/arXiv.2503.23571. Används för datainsamling, episodhantering och återställning som designinspiration. Projektet reproducerar inte experimenten eller resultaten.

[S07] Nowaste Logistics Advances Automation with Cognibotics Robot Picking Cell

Leverantörens installationsbeskrivning

Cognibotics. 5 januari 2026. Kontrollerad 2026-10-01.

<https://www.cognibotics.com/en/articles/nowaste-logistics-advances-automation-with-cognibotics-robot-picking-cell>

Beskriver andra cellen, AutoStore, SICS AI-vision och Juliet & Romeo. Leverantörsuppgifter, inte publicerad intern API-specifikation eller oberoende driftsrevision.

[S10] SICS AI: HPC Access to LUMI Supercomputer for AI and Robotics

Teknisk samarbetsbeskrivning

ENCCS. 12 september 2025. Kontrollerad 2026-10-01.

<https://enccs.se/2025/09/news/sics-hpc-access-lumi-supercomputer-ai-robotics-3/>

Dokumenterar arbete med beräkningsmiljö och GPU-träning. Fastställer inte kundernas aktuella inferensarkitektur.

[S11] SICS AI investerar i NVIDIA DGX B200 från Aixa

Leverantörens pressmeddelande

Aixa Group AB. 17 juni 2025. Kontrollerad 2026-10-01.

<https://news.cision.com/se/aixa-group-ab--publ-/r/sics-ai-investerar-i-nvidia-dgx-b200-fran-aixa---accelererar-utvecklingen-av-ai-hjarna-for-nasta-ge%2Cc4165473>

Bekräftar offentliggjord DGX B200-affär med drifttjänster. Inte ett test av modeller eller belägg för var dagens maskin är placerad.

[S20] About Us

Företagets egen presentation

SICS AI. Utan datum. Kontrollerad 2026-10-01.

<https://sics.ai/about/>

Används endast för den uppgivna rollen Chief AI Architect för Per-Eric Olsson. Övriga karriär- och AGI-påståenden övertas inte som verifierade meriter.
